



Bericht in

Entwicklung einer Metaheuristik für das Job Shop Scheduling Problem unter Berücksichtigung von reihenfolgeabhängigen Rüstzeiten

Jakob Weber Matr.-Nr. 3869088
Johannes Walz Matr.-Nr. 3533824

Prüfer: Prof. Dr. David Müller

© Jakob Weber Matr.-Nr. 3869088, Johannes Walz Matr.-Nr. 3533824, 2026

Diese Arbeit, einschließlich ihrer Teile, **ist urheberrechtlich geschützt**. Jede Verwertung außerhalb der engen Grenzen des Urheberrechtsgesetzes ohne Zustimmung von den Urhebern ist untersagt und strafbar. Dies gilt insbesondere für Vervielfältigungen, Übersetzungen, Mikroverfilmung und die Speicherung und Verarbeitung in elektronischen Systemen.

Inhaltsverzeichnis

1	Einleitung	2
1.1	Problemstellung	2
1.2	Zielsetzung	2
1.3	Aufbau der Arbeit	2
2	Notation	3
3	Methodik	5
3.1	Eröffnungsheuristik - Giffler-Thompson-Algorithmus	5
3.2	Metaheuristik - Tabu-Suche	5
3.2.1	Kritischer Pfad und kritische Blöcke	6
3.2.2	Nachbarschaftsdefinitionen	7
3.3	Hardwarespezifikationen	8
4	Ergebnisse	9
5	Diskussion	10

Kapitel 1

Einleitung

1.1 Problemstellung

1.2 Zielsetzung

Wie beeinflusst die Wahl der Nachbarschaftsdefinition in einer Tabu Suche den Makespan von Job-Shop-Scheduling-Problemen mit reihenfolgeabhängigen Rüstzeiten?

1.3 Aufbau der Arbeit

Kapitel 2

Notation

Notation der Job-Shop-Scheduling-Problems mit sequenzabhängigen Rüstzeiten

In diesem Kapitel wird die Notation für das betrachtete Job-Shop-Scheduling-Problem mit sequenzabhängigen Rüstzeiten eingeführt. Die Notation dient als Grundlage für die Beschreibung der Tabu-Search-Heuristik und der verwendeten Bewertungslogik.

Gegeben ist eine Menge von Jobs J und eine Menge von Maschinen M . Jeder Job $j \in J$ besteht aus einer geordneten Folge von Operationen

$$O_j = (o_{j,1}, o_{j,2}, \dots, o_{j,q_j}).$$

Die Reihenfolge der Operationen innerhalb eines Jobs ist vorgegeben. Eine Operation $o_{j,k}$ darf daher erst beginnen, wenn ihre direkte Vorgängeroperation $o_{j,k-1}$ abgeschlossen ist.

Jede Operation $o_{j,k}$ wird auf genau einer Maschine $m_{j,k} \in M$ bearbeitet und besitzt eine Bearbeitungszeit $p_{j,k}$. Zwischen zwei Operationen, die auf derselben Maschine direkt aufeinanderfolgen, kann eine sequenzabhängige Rüstzeit $s_{u,v}$ anfallen. Diese hängt von der vorherigen Operation u und der nachfolgenden Operation v ab.

Tabelle 2.1: Notation des Job-Shop-Scheduling-Problems

Symbol	Bedeutung
J	Menge der Jobs
j	Index eines Jobs, $j \in J$
M	Menge der Maschinen
m	Index einer Maschine, $m \in M$
O_j	geordnete Menge der Operationen von Job j
$o_{j,k}$	k -te Operation von Job j
q_j	Anzahl der Operationen von Job j
O	Menge aller Operationen
$m_{j,k}$	Maschine, auf der Operation $o_{j,k}$ bearbeitet wird
$p_{j,k}$	Bearbeitungszeit der Operation $o_{j,k}$
$s_{u,v}$	Rüstzeit zwischen zwei direkt aufeinanderfolgenden Operationen u und v
$t(o)$	Startzeit der Operation o
$C(o)$	Endzeit der Operation o
$C_{\max}$	Makespan einer Lösung
$F(S)$	Zielfunktionswert der Lösung S

Notation der Tabu-Suche

Zur Lösung des Problems wird eine Tabu-Search-Heuristik verwendet. Die aktuelle Lösung wird mit S bezeichnet. Die beste bisher gefundene Lösung wird als S^* notiert. Die Nachbarschaft der aktuellen Lösung wird durch $NB(S)$ beschrieben. Sie enthält alle Lösungen, die durch zulässige lokale Bewegungen aus S erzeugt werden können.

Die Tabu-Liste TL speichert Bewegungsattribute, die für eine begrenzte Anzahl von Iterationen tabu sind. Dadurch wird ein unmittelbares Zurückkehren in vorherige Lösungen verhindert. Tabuierte Bewegungen können durch das Aspirationskriterium zugelassen werden, wenn sie zu einer neuen globalen Bestlösung führen.

Tabelle 2.2: Notation der Tabu-Search-Heuristik

Symbol	Bedeutung
$\hat{S}$	erzeugte Startlösung
S	aktuelle Lösung
S^*	beste bisher gefundene Lösung
S'	Nachbarlösung der aktuellen Lösung
$NB(S)$	Nachbarschaft der aktuellen Lösung S
TL	Tabu-Liste
TL^{Dauer}	Gültigkeitsdauer eines Tabu-Eintrags
I	Anzahl der Iterationen ohne Verbesserung
$I^{\max}$	maximale Anzahl an Iterationen ohne Verbesserung
I_{total}	Gesamtzahl der Iterationen
$I_{\text{total}}^{\max}$	maximale Gesamtzahl an Iterationen
$I^{\text{Diversifikation}}$	Schwelle für zusätzliche Diversifikationsbewegungen
$\tau^{\max}$	optionales Zeitlimit
$F(S)$	Zielfunktionswert der Lösung S

Da der Makespan minimiert wird, gilt äquivalent:

$$C_{\max}(S') < C_{\max}(S^*).$$

Kapitel 3

Methodik

3.1 Eröffnungsheuristik - Giffler-Thompson-Algorithmus

3.2 Metaheuristik - Tabu-Suche

Algorithm 1 Tabu Search für das Job-Shop-Scheduling-Problem mit sequenzabhängigen Rüstzeiten

```
1:  $S \leftarrow \hat{S}$ 
2:  $S^* \leftarrow S$ 
3:  $TL \leftarrow \emptyset$ 
4:  $I_{\text{total}} \leftarrow 0$ 
5:  $I \leftarrow 0$ 
6:  $\tau \leftarrow 0$ 
7: while  $I_{\text{total}} < I_{\text{total}}^{\max}$  and  $I < I^{\max}$  and  $\tau < \tau^{\max}$  do
8:    $I_{\text{total}} \leftarrow I_{\text{total}} + 1$ 
9:   Erzeuge die Nachbarschaft  $NB(S)$  der aktuellen Lösung
10:  if  $I \geq I^{\text{div}}$  then
11:    Ergänze zufällige nicht-kritische Bewegungen zu  $NB(S)$ 
12:  end if
13:  for  $S' \in NB(S)$  do (jede Nachbarlösung)
14:    if  $S'$  ist unzulässig or (tabuiert and Aspiration nicht erfüllt) then
15:      Entferne  $S'$  aus  $NB(S)$ 
16:    end if
17:  end for
18:  Wähle die beste zulässige Nachbarlösung  $S' \in NB(S)$ 
19:   $S \leftarrow S'$ 
20:  Entferne abgelaufene Einträge aus  $TL$ 
21:   $TL \leftarrow TL \cup \{\text{inverse Bewegung von } S'\}$ 
22:  if  $F(S) < F(S^*)$  then
23:     $S^* \leftarrow S$ 
24:     $I \leftarrow 0$ 
25:  else
26:     $I \leftarrow I + 1$ 
27:  end if
28: end while
29: return  $S^*$ 
```

Die Suche beginnt mit der Erzeugung einer zulässigen Startlösung (S). Anschließend wird die beste gefundene Lösung mit ($\hat{S} = S$) initialisiert. Danach werden die Tabu-Liste (TL), der Gesamtiterationenzähler (I^{total}) und der Zähler für Iterationen ohne Verbesserung (I) initialisiert. Die Suche wird fortgeführt, bis ein Abbruchkriterium erfüllt ist. Der Gesamtiterationenzähler wird in jedem Schleifendurchlauf um eins erhöht.

Der Algorithmus verwendet drei Abbruchkriterien. Erstens endet die Suche nach Erreichen der maximalen Gesamtzahl an Iterationen. Zweitens endet sie nach einer festgelegten Anzahl an Iterationen ohne Verbesserung. Drittens kann ein Zeitlimit gesetzt werden. Der Timer wird nur aktiviert, wenn beim Aufruf des Algorithmus ein Zeitlimit übergeben wird.

In jeder Iteration wird zunächst die Nachbarschaft ($NB(S)$) der aktuellen Lösung erzeugt. Ergänzend werden potenzielle Kandidatenbewegungen gebildet. Wird über mehrere Iterationen keine Verbesserung erzielt, werden zusätzliche zufällige nicht-kritische Bewegungen ergänzt. Diese dienen der Diversifikation der Suche. Liegen mehrere zulässige Kandidaten mit identischem Makespan vor, werden diese zufällig durchmischt. Dadurch soll ein Festlaufen in lokalen Optima reduziert werden.

Tabuierte Bewegungen können trotzdem verwendet werden, sofern die Aspirationsbedingung erfüllt ist. Diese ist erfüllt, wenn eine tabuierte Bewegung zu einer Verbesserung der gesamten Laufzeit führt. Aus den verbleibenden Kandidaten wird die beste zulässige Nachbarlösung (S') ausgewählt. Existiert keine zulässige Nachbarlösung, wird die Suche beendet.

Wird eine Nachbarlösung akzeptiert, wird die aktuelle Lösung durch (S') ersetzt. Anschließend wird die Tabu-Liste mit der inversen Bewegung aktualisiert. Dadurch wird ein direktes Zurückkehren in den vorherigen Zustand verhindert. Ist die neue Lösung besser als die bisher beste Lösung, gilt also ($F(S) < F(S^*)$), wird (S^*) durch (S) ersetzt und der Verbesserungszähler zurückgesetzt. Andernfalls wird der Verbesserungszähler erhöht. Nach Abschluss der Suche wird die beste gefundene Lösung (S^*) zurückgegeben.

3.2.1 Kritischer Pfad und kritische Blöcke

Die Bewertung einer Lösung erfolgt auf Basis eines disjunktiven Graphen. Jede Operation wird durch einen Knoten repräsentiert. Zwischen den Knoten existieren konjunktive Kanten für die technologische Reihenfolge innerhalb eines Jobs sowie disjunktive Kanten für die Bearbeitungsreihenfolge auf einer Maschine. Die konjunktiven Kanten sind durch die Instanz vorgegeben. Ihre Gewichte entsprechen den Bearbeitungszeiten der zugehörigen Operationen. Die Richtung der disjunktiven Kanten ist zunächst offen und wird erst durch die in der aktuellen Lösung festgelegten Maschinenreihenfolgen bestimmt. Auch diese Kanten tragen die Bearbeitungszeit und gegebenenfalls zusätzliche sequenzabhängige Rüstzeiten. Sobald für jede Maschine eine zulässige Reihenfolge vorliegt, ist der resultierende Graph azyklisch.

In diesem azyklischen Graphen entspricht die früheste Startzeit einer Operation der Länge des längsten Pfades von einer Quelle zu diesem Knoten. Die Funktion `evaluate_orders(problem, orders)` bestimmt diese Werte mit einer um eine Longest-Path-Berechnung. Zunächst wird für jede Operation der Eingangsgrad bestimmt. Dabei besitzt eine Operation höchstens zwei unmittelbare Vorgänger, nämlich einen Job-Vorgänger und einen Maschinen-Vorgänger. Anschließend werden alle Operationen ohne Vorgänger mit Startzeit null in eine Warteschlange eingefügt. Wird eine Operation u verarbeitet, werden ihre möglichen Nachfolger entlang beider Kantentypen relaxiert. Für jeden Nachfolger v gilt

$$start(v) = \max(start(v), completion(u) + s_{uv}),$$

wobei s_{uv} die zusätzliche Rüstzeit auf der betrachteten Maschinenkante bezeichnet und auf konjunktiven Kanten null ist. Die Maximumbildung ist erforderlich, weil eine Operation erst beginnen darf, wenn alle unmittelbaren Vorgänger abgeschlossen sind. Immer dann, wenn eine Relaxation den Startzeitwert eines Nachfolgers erhöht, wird der zugehörige Vorgänger als bindender Vorgänger gespeichert. Dieser Zeiger identifiziert genau die Kante, die die aktuelle früheste Startzeit bestimmt.

Werden im Verlauf der topologischen Verarbeitung nicht alle Operationen besucht, enthält der disjunktive Graph einen Zyklus. Eine solche Reihenfolge ist unzulässig und wird verworfen. Nach Abschluss der Berechnung ergibt sich der Makespan $C_{\max}$ als maximale Fertigstellungszeit über alle Operationen. Der kritische Pfad wird anschließend durch Rückverfolgung der gespeicherten bindenden Vorgänger konstruiert. Ausgehend von einer Operation mit Fertigstellungszeit $C_{\max}$ wird iterativ zum jeweiligen Vorgänger zurückgegangen, bis eine Quelle erreicht ist. Die umgekehrte Folge dieser Operationen bildet einen kritischen Pfad. Die Summe seiner Kantengewichte entspricht exakt dem Makespan.

Auf dieser Grundlage werden kritische Blöcke bestimmt. Ein kritischer Block ist eine maximale Teilfolge des kritischen Pfades, deren Operationen auf derselben Maschine liegen und dort unmittelbar aufeinanderfolgen. Diese Blöcke definieren den Suchraum der kritischkeitsbasierten Nachbarschaften.

Die aktuelle Implementierung liefert bei mehreren gleich langen kritischen Pfaden nur einen einzelnen Pfad zurück. Dies folgt erstens aus der Wahl des Endknotens, da bei mehreren Operationen mit identischer maximaler Fertigstellungszeit nur der erste gefundene Endpunkt verwendet wird. Zweitens wird bei Gleichstand konkurrierender Vorgänger nur die zuerst verarbeitete bindende Kante gespeichert. Der berechnete Makespan bleibt dadurch korrekt. Eingeschränkt ist jedoch die nachgelagerte Nachbarschaftserzeugung, weil kritische Blöcke ausschließlich aus diesem einen Pfad extrahiert werden. Existieren mehrere voneinander unabhängige kritische Pfade, können erzeugte Bewegungen einen sichtbaren Pfad verändern, ohne den Makespan zu reduzieren, weil ein weiterer unbeachteter Pfad weiterhin bindend bleibt. Im vorliegenden Verfahren wird dieser Effekt nur teilweise abgeschwächt. Zusätzliche nicht-kritische Diversifikationszüge können nach längerer Stagnation indirekt einen weiteren kritischen Pfad treffen. Außerdem kann sich durch eine geänderte Reihenfolge in einer späteren Iteration ein anderer gleich langer Pfad als kritisch manifestieren. Eine vollständige Behandlung würde die gleichzeitige Identifikation aller Endoperationen mit Fertigstellungszeit $C_{\max}$ und die Berücksichtigung mehrerer kritischer Pfade in der Blockbildung erfordern.

3.2.2 Nachbarschaftsdefinitionen

Für die Tabu-Suche werden drei Nachbarschaftsdefinitionen verwendet, die auf dem kritischen Pfad basieren. Der kritische Pfad ist die Folge von Operationen, die den Makespan $C_{\max}$ bestimmt. Eine Verzögerung einer Operation auf diesem Pfad erhöht unmittelbar die Gesamtdauer. [1]

Zur Erzeugung von Bewegungen wird der kritische Pfad in kritische Blöcke zerlegt. Ein kritischer Block ist eine maximale Teilfolge von Operationen, die auf dem kritischen Pfad liegen und auf derselben Maschine unmittelbar aufeinanderfolgen. Die Funktion `critical_blocks()` durchläuft den kritischen Pfad und fasst Operationen so lange zu einem Block zusammen, wie Maschinenzuordnung und unmittelbare Nachbarschaft erhalten bleiben. Bei Maschinenwechsel oder fehlender Adjazenz wird ein neuer Block begonnen. Blöcke mit weniger als zwei Operationen werden verworfen, da kein sinnvoller Tausch definiert ist. [PLATZHALTER: Zitat Konzept kritische Blöcke, z.B. Nowicki & Smutnicki 1996]

Die Einschränkung auf Operationen innerhalb kritischer Blöcke folgt der klassischen Eigenschaft, dass Verbesserungen des Makespans nur durch Änderungen innerhalb solcher Blöcke erreicht werden. Vertauschungen über Blockgrenzen hinweg reduzieren den Makespan nicht. [PLATZHALTER: Zitat, z.B. van Laarhoven, Aarts & Lenstra 1992]

Die Implementierung umfasst drei Nachbarschaften.

Adjacent-Swap-Nachbarschaft. Für jeden kritischen Block werden alle benachbarten Operationspaare vertauscht. Ein Block $[A, B, C, D]$ erzeugt die Züge (A, B) , (B, C) und (C, D) . Diese Variante betrachtet alle lokalen Adjazenzvertauschungen innerhalb kritischer Blöcke. [PLATZHALTER: Zitat]

N5-Nachbarschaft. Pro kritischem Block werden nur die ersten beiden sowie die letzten beiden Operationen vertauscht. Ein Block $[A, B, C, D, E]$ erzeugt damit ausschließlich die Züge (A, B) und (D, E) . Die Reduktion basiert auf dem Ergebnis, dass Randvertauschungen makespanwirksam sein

können, während innere Vertauschungen diese Eigenschaft nicht besitzen. [PLATZHALTER: Zitat N5, z.B. Nowicki & Smutnicki 1996, "A Fast Taboo Search Algorithm for the Job Shop Problem"]

N6-Nachbarschaft. N6 erweitert N5 um Insert-Bewegungen. Zusätzlich zu den N5-Randvertauschungen kann jede innere Operation eines Blocks an den Blockanfang oder an das Blockende verschoben werden. Für den Block $[A, B, C, D, E]$ entstehen neben (A, B) und (D, E) beispielsweise die Sequenzen $[C, A, B, D, E]$ und $[A, C, D, E, B]$. Duplikate werden ausgeschlossen, wenn ein Insert-Zug äquivalent zu einem bereits enthaltenen N5-Zug ist. Diese Erweiterung vergrößert die strukturelle Diversität der Nachbarschaft bei weiterhin begrenzter Kandidatenzahl. [PLATZHALTER: Zitat N6, z.B. Balas & Vazacopoulos 1998, "Guided local search with shifting bottleneck for job shop scheduling"]

3.3 Hardwarespezifikationen

Die Instanzen wurden auf einem Apple MacBook Pro aus dem Jahr 2023 ausgeführt. Das System ist mit einem Apple M3 Pro Prozessor sowie 18 GB gemeinsam genutztem Arbeitsspeicher ausgestattet. Die Berechnungen wurden lokal auf dem Gerät durchgeführt, ohne zusätzliche externe Rechenressourcen oder Serverinfrastruktur zu verwenden.

Kapitel 4

Ergebnisse

Tabelle 1: Ergebnisse der Giffler-Thompson-Prioritätsregeln

- Prioritätsregel- Cmax- Laufzeit- Anzahl Rüstvorgänge oder gesamte Rüstzeit, falls relevant

Tabelle 2: Vergleich der Tabu-Search-Nachbarschaften

- Nachbarschaft- bester Cmax - durchschnittlicher Cmax- schlechtester Cmax - Standardabweichung- durchschnittliche Laufzeit- Anzahl Iterationen

Tabelle 3: Vergleich mit OR-Tools

- Instanz - OR-Tools Cmax - Tabu Search Cmax - absolute Abweichung - relative Abweichung in % - Laufzeit OR-Tools - Laufzeit Tabu Search

Kapitel 5

Diskussion

Abbildungsverzeichnis

Quellcodeverzeichnis

Tabellenverzeichnis

2.1	Notation des Job-Shop-Scheduling-Problems	3
2.2	Notation der Tabu-Search-Heuristik	4

Literaturverzeichnis

- [1] J. Blazewicz, K. H. Ecker, E. Pesch, G. Schmidt, M. Sterna, und J. Weglarz, *Handbook on Scheduling: From Theory to Practice*. Springer International Publishing. [Online]. Available: <http://link.springer.com/10.1007/978-3-319-99849-7> (Zitiert auf Seite 7.)